

Gesture Recognition Web Interface using PHP and MediaPipe



Intended Learning Outcomes (ILO)

After completing this laboratory activity, students will be able to:

1. Design and organize a web-based interface using HTML, CSS, and PHP.
2. Display real-time system feedback for webcam-based applications.
3. Improve usability through proper labeling, instructions, and control design.

Objectives

By the end of this activity, students should be able to:

1. Develop a clean and organized interface for a gesture recognition application.
2. Display detected gestures clearly within the web interface.
3. Design functional camera control buttons.
4. Apply responsive layout principles for different screen sizes.

What is Gesture Recognition?

Gesture recognition is a computer vision technique that identifies hand movements using a webcam. MediaPipe detects hand landmarks which allow gestures such as Open Palm, Closed Fist, Peace Sign, and Thumbs Up to be recognized.

What is MediaPipe Hands?

MediaPipe Hands detects 21 hand landmark points. These landmarks help determine whether fingers are extended or folded to classify gestures.

Instructions

Step 1: Setup the Project

1. **Start XAMPP Control Panel.**
2. **Start Apache Server.**
3. **Project folder into:** xampp/htdocs/gesture-recognition/

Step 2: Open the Project

1. Open Visual Studio Code.
2. Open the folder: gesture-recognition
3. Navigate to: index.php

Reference Code:

```
<?php ?>
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,initial-scale=1.0" />
  <title>Gesture Recognition</title>

  <style>
    body {
      margin: 0;
      font-family: Segoe UI, Arial;
      background: #0f172a;
      color: #fff;
    }

    .container {
      max-width: 1000px;
      margin: auto;
```

```
padding: 20px;
}

.stage {
  position: relative;
  aspect-ratio: 16/9;
  background: #000;
  border-radius: 12px;
  overflow: hidden;
  border: 1px solid #334155;
}

#stream {
  position: absolute;
  inset: 0;
  width: 100%;
  height: 100%;
  object-fit: cover;
}

.hud {
  position: absolute;
  top: 15px;
  left: 15px;
  background: rgba(0, 0, 0, .6);
  padding: 12px;
  border-radius: 10px;
  min-width: 220px;
}

.status {
  font-weight: 900;
  font-size: 18px;
}

.good {
  color: #22c55e;
}

.warn {
  color: #f59e0b;
}

.bad {
  color: #ef4444;
```

```

    }

    .controls {
        margin-top: 10px;
        display: flex;
        gap: 10px;
        flex-wrap: wrap;
    }

    button {
        padding: 10px 14px;
        border-radius: 10px;
        border: none;
        font-weight: bold;
        cursor: pointer;
        background: #1e293b;
        color: #fff;
    }

    button:hover {
        background: #334155;
    }

    .note {
        margin-top: 10px;
        color: #cbd5e1;
        font-size: 14px;
        line-height: 1.4;
    }

    code {
        background: #111827;
        padding: 2px 6px;
        border-radius: 6px;
    }
</style>
</head>

<body>
    <div class="container">
        <h2>Gesture Recognition</h2>

        <div class="stage">
            <img id="stream" alt="Camera Stream" />

```

```

    <div class="hud">
        <div>System</div>
        <div id="sysText" class="status warn">READY</div>

        <div style="margin-top:10px">Gesture</div>
        <div id="gestureText" class="status warn">---</div>
    </div>
</div>

<div class="controls">
    <button id="startBtn">Start Camera</button>
    <button id="stopBtn">Stop</button>
    <button id="checkBtn">Check Backend</button>
</div>

<div class="note">
    Required workflow:<br>
    1) Run <code>python gesture_recognition.py</code> (leave terminal
open)<br>
    2) Open this page:
<code>http://localhost/yourfolder/index.php</code><br>
    3) Click <b>Start Camera</b>
</div>
</div>

<script>
    // If you changed Python port, update here:
    const PY_BASE = "http://127.0.0.1:5000";

    const streamImg = document.getElementById("stream");
    const sysText = document.getElementById("sysText");
    const gestureText = document.getElementById("gestureText");

    let pollTimer = null;

    function setSystem(txt, type = "warn") {
        sysText.innerText = txt;
        sysText.className = "status " + type;
    }

    function setGesture(txt, type = "warn") {
        gestureText.innerText = txt;
        gestureText.className = "status " + type;
    }

```

```

async function getJSON(url) {
  const res = await fetch(url, { cache: "no-store" });
  if (!res.ok) throw new Error("HTTP " + res.status);
  return res.json();
}

async function postJSON(url) {
  const res = await fetch(url, { method: "POST" });
  if (!res.ok) throw new Error("HTTP " + res.status);
  return res.json();
}

async function checkBackend() {
  try {
    const data = await getJSON(PY_BASE + "/health");
    setSystem(data.running ? "BACKEND OK (RUNNING)" : "BACKEND OK (IDLE)", "good");
    return true;
  } catch (e) {
    setSystem("PYTHON BACKEND NOT RUNNING", "bad");
    return false;
  }
}

async function startSystem() {
  const ok = await checkBackend();
  if (!ok) {
    setGesture("---", "warn");
    streamImg.removeAttribute("src");
    return;
  }

  try {
    setSystem("STARTING...", "warn");
    await postJSON(PY_BASE + "/start");

    // Load MJPEG stream (cache-bust)
    streamImg.src = PY_BASE + "/video_feed?ts=" + Date.now();
    setSystem("RUNNING", "good");

    if (pollTimer) clearInterval(pollTimer);
    pollTimer = setInterval(async () => {
      try {
        const g = await getJSON(PY_BASE + "/gesture");
        const label = g.gesture || "---";

```

```

        const type =
            (label === "UNKNOWN") ? "warn" :
            (label === "NO HAND") ? "bad" :
            (label === "STOPPED") ? "warn" : "good";
        setGesture(label, type);
    } catch (e) {
        setSystem("BACKEND LOST", "bad");
        setGesture("---", "warn");
    }
}, 200);

} catch (e) {
    setSystem("FAILED TO START", "bad");
    setGesture("---", "warn");
}
}

async function stopSystem() {
    try { await postJSON(PY_BASE + "/stop"); } catch (e) { }
    if (pollTimer) clearInterval(pollTimer);
    pollTimer = null;

    streamImg.removeAttribute("src");
    setSystem("STOPPED", "warn");
    setGesture("---", "warn");
}

document.getElementById("startBtn").onclick = startSystem;
document.getElementById("stopBtn").onclick = stopSystem;
document.getElementById("checkBtn").onclick = checkBackend;

// optional: auto-check on load
checkBackend();
</script>
</body>
</html>

```

4. Navigate to: `gesture_recognition.py`

Install requirements (terminal) –

```
pip install flask flask-cors opencv-python mediapipe numpy
```

IF YOU ARE USING WINDOWS use this code, if not-- keep scrolling for Mac

```
gesture_recognition.py X index.php
gesture_recognition.py > ...
1  import cv2
2  import mediapipe as mp
3  import math
4  import time
5  import numpy as np
6  from threading import Lock
7  from flask import Flask, Response, jsonify
8  from flask_cors import CORS
9
10 app = Flask(__name__)
11 CORS(app)
12
13 mp_hands = mp.solutions.hands
14 mp_draw = mp.solutions.drawing_utils
15
16 lock = Lock()
17 cap = None
18 running = False
19 latest_label = "READY"
20
```


gesture_recognition.py X index.php

gesture_recognition.py > ...

```
21
22 # ----- Helpers -----
23 def dist(a, b):
24     return math.hypot(a.x - b.x, a.y - b.y)
25
26 def finger_extended(hand, tip_id, pip_id, mcp_id):
27     wrist = hand.landmark[0]
28     tip = hand.landmark[tip_id]
29     pip = hand.landmark[pip_id]
30     mcp = hand.landmark[mcp_id]
31     return dist(tip, wrist) > dist(pip, wrist) > dist(mcp, wrist)
32
33 def thumb_extended(hand):
34     wrist = hand.landmark[0]
35     thumb_tip = hand.landmark[4]
36     thumb_ip = hand.landmark[3]
37     return dist(thumb_tip, wrist) > dist(thumb_ip, wrist) * 1.05
38
39 def finger_folded(hand, tip_id, pip_id):
40     wrist = hand.landmark[0]
41     tip = hand.landmark[tip_id]
42     pip = hand.landmark[pip_id]
43     return dist(tip, wrist) < dist(pip, wrist)
44
45 def recognize_gesture(hand):
46     thumb = thumb_extended(hand)
47     index = finger_extended(hand, 8, 6, 5)
48     middle = finger_extended(hand, 12, 10, 9)
49     ring = finger_extended(hand, 16, 14, 13)
50     pinky = finger_extended(hand, 20, 18, 17)
51
52     index_fold = finger_folded(hand, 8, 6)
53     middle_fold = finger_folded(hand, 12, 10)
54     ring_fold = finger_folded(hand, 16, 14)
55     pinky_fold = finger_folded(hand, 20, 18)
56
57     if thumb and index and middle and ring and pinky:
58         return "OPEN PALM"
59
60     if (index_fold or not index) and (middle_fold or not middle) and (ring_fold or not ring) and (pinky_fold or not pinky) and (not thumb):
61         return "CLOSED FIST"
62
63     if index and middle and (ring_fold or not ring) and (pinky_fold or not pinky):
64         return "PEACE SIGN"
65
66     if thumb and (index_fold or not index) and (middle_fold or not middle) and (ring_fold or not ring) and (pinky_fold or not pinky):
67         return "THUMBS UP"
68
69     return "UNKNOWN"
```

```

71
72 # ----- Camera control -----
73 def ensure_camera_open():
74     global cap
75     if cap is None:
76         cap = cv2.VideoCapture(0, cv2.CAP_DSHOW) # CAP_DSHOW helps on Windows
77         # Optional: set resolution
78         cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
79         cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
80
81 def release_camera():
82     global cap
83     if cap is not None:
84         cap.release()
85         cap = None
86
87 def blank_frame(width=1280, height=720, text="STOPPED"):
88     frame = np.zeros((height, width, 3), dtype=np.uint8)
89     cv2.putText(frame, text, (40, 90),
90                 cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 255, 255), 3)
91     return frame
92

```

```

93
94 # ----- MJPEG stream -----
95 def generate_frames():
96     global latest_label, running
97
98     with mp_hands.Hands(
99         static_image_mode=False,
100         max_num_hands=1,
101         model_complexity=1,
102         min_detection_confidence=0.6,
103         min_tracking_confidence=0.6
104     ) as hands:
105
106         while True:
107             if not running:
108                 latest_label = "STOPPED"
109                 frame = blank_frame(text="STOPPED - Click Start")
110                 ok, buf = cv2.imencode(".jpg", frame)
111                 if ok:
112                     yield (b"--frame\r\n"
113                           + b"Content-Type: image/jpeg\r\n\r\n" + buf.tobytes() + b"\r\n")
114                 time.sleep(0.08)
115                 continue
116

```

```

117     with lock:
118         ensure_camera_open()
119         ok, frame = cap.read()
120
121     if not ok or frame is None:
122         latest_label = "CAMERA ERROR"
123         frame = blank_frame(text="CAMERA ERROR")
124         ok2, buf2 = cv2.imencode(".jpg", frame)
125         if ok2:
126             yield (b"--frame\r\n"
127                   | b"Content-Type: image/jpeg\r\n\r\n" + buf2.tobytes() + b"\r\n")
128         time.sleep(0.1)
129         continue
130
131     frame = cv2.flip(frame, 1)
132     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
133     res = hands.process(rgb)
134
135     label = "NO HAND"
136     if res.multi_hand_landmarks:
137         hand_landmarks = res.multi_hand_landmarks[0]
138         label = recognize_gesture(hand_landmarks)
139         mp_draw.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
140
141     latest_label = label
142
143     # overlay label (so students see it even if polling fails)
144     cv2.rectangle(frame, (10, 10), (420, 70), (0, 0, 0), -1)
145     cv2.putText(frame, f"Gesture: {latest_label}", (20, 55),
146               cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 255, 255), 2)
147
148     ok3, buf3 = cv2.imencode(".jpg", frame)
149     if not ok3:
150         continue

```

```

151
152     yield (b"--frame\r\n"
153           | b"Content-Type: image/jpeg\r\n\r\n" + buf3.tobytes() + b"\r\n")
154
155

```

```

155
156 # ----- API routes -----
157 @app.get("/health")
158 def health():
159     return jsonify({"ok": True, "running": running})
160
161 @app.post("/start")
162 def start():
163     global running, latest_label
164     running = True
165     latest_label = "RUNNING"
166     return jsonify({"ok": True, "running": running})
167
168 @app.post("/stop")
169 def stop():
170     global running, latest_label
171     running = False
172     latest_label = "STOPPED"
173     with lock:
174         release_camera()
175     return jsonify({"ok": True, "running": running})
176
177 @app.get("/gesture")
178 def gesture():
179     return jsonify({"gesture": latest_label, "running": running})
180
181 @app.get("/video_feed")
182 def video_feed():
183     return Response(generate_frames(), mimetype="multipart/x-mixed-replace; boundary=frame")
184
185
186 if __name__ == "__main__":
187
188     app.run(host="127.0.0.1", port=5000, debug=False, threaded=True)

```

USING MAC

gesture_recognition.py

```
C: > Users > Khae > Downloads > 📁 gesture_recognition.py > ...
1  import cv2
2  import mediapipe as mp
3  import math
4  import time
5  import numpy as np
6  import sys
7  from threading import Lock
8  from flask import Flask, Response, jsonify
9  from flask_cors import CORS
10
11  app = Flask(__name__)
12  CORS(app)
13
14  mp_hands = mp.solutions.hands
15  mp_draw = mp.solutions.drawing_utils
16
17  lock = Lock()
18  cap = None
19  running = False
20  latest_label = "READY"
21
```

```

23 # ----- Helpers -----
24 def dist(a, b):
25     return math.hypot(a.x - b.x, a.y - b.y)
26
27 def finger_extended(hand, tip_id, pip_id, mcp_id):
28     wrist = hand.landmark[0]
29     tip = hand.landmark[tip_id]
30     pip = hand.landmark[pip_id]
31     mcp = hand.landmark[mcp_id]
32     return dist(tip, wrist) > dist(pip, wrist) > dist(mcp, wrist)
33
34 def thumb_extended(hand):
35     wrist = hand.landmark[0]
36     thumb_tip = hand.landmark[4]
37     thumb_ip = hand.landmark[3]
38     return dist(thumb_tip, wrist) > dist(thumb_ip, wrist) * 1.05
39
40 def finger_folded(hand, tip_id, pip_id):
41     wrist = hand.landmark[0]
42     tip = hand.landmark[tip_id]
43     pip = hand.landmark[pip_id]
44     return dist(tip, wrist) < dist(pip, wrist)
45
46 def recognize_gesture(hand):
47     thumb = thumb_extended(hand)
48     index = finger_extended(hand, 8, 6, 5)
49     middle = finger_extended(hand, 12, 10, 9)
50     ring = finger_extended(hand, 16, 14, 13)
51     pinky = finger_extended(hand, 20, 18, 17)
52

```

```

53     index_fold = finger_folded(hand, 8, 6)
54     middle_fold = finger_folded(hand, 12, 10)
55     ring_fold = finger_folded(hand, 16, 14)
56     pinky_fold = finger_folded(hand, 20, 18)
57
58     if thumb and index and middle and ring and pinky:
59         return "OPEN PALM"
60
61     if (index_fold or not index) and (middle_fold or not middle) and (ring_fold or not ring) and (pinky_fold or not pinky) and (not thumb):
62         return "CLOSED FIST"
63
64     if index and middle and (ring_fold or not ring) and (pinky_fold or not pinky):
65         return "PEACE SIGN"
66
67     if thumb and (index_fold or not index) and (middle_fold or not middle) and (ring_fold or not ring) and (pinky_fold or not pinky):
68         return "THUMBS UP"
69
70     return "UNKNOWN"
71

```

```

72
73 # ----- Camera control -----
74 def ensure_camera_open():
75     """
76     Cross-platform camera open:
77     - Windows: CAP_DSHOW
78     - macOS: CAP_AVFOUNDATION
79     - Linux: default
80     """
81     global cap
82
83     if cap is None:
84         if sys.platform.startswith("win"):
85             cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
86         elif sys.platform == "darwin":
87             cap = cv2.VideoCapture(0, cv2.CAP_AVFOUNDATION)
88         else:
89             cap = cv2.VideoCapture(0)
90
91         # Give macOS a short time to initialize camera
92         time.sleep(0.2)
93
94         # Optional resolution
95         cap.set(cv2.CAP_PROP_FRAME_WIDTH, 1280)
96         cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 720)
97
98         # Validate
99         if not cap.isOpened():
100             try:
101                 cap.release()
102             except Exception:
103                 pass
104             cap = None
105

```

```

106
107 def release_camera():
108     global cap
109     if cap is not None:
110         try:
111             cap.release()
112         except Exception:
113             pass
114         cap = None
115
116
117 def blank_frame(width=1280, height=720, text="STOPPED"):
118     frame = np.zeros((height, width, 3), dtype=np.uint8)
119     cv2.putText(frame, text, (40, 90),
120                 cv2.FONT_HERSHEY_SIMPLEX, 2, (255, 255, 255), 3)
121     return frame
122

```

```

C: > Users > Khae > Downloads > gesture_recognition.py > blank_frame
122
123
124 # ----- MJPEG stream -----
125 def generate_frames():
126     global latest_label, running
127
128     with mp_hands.Hands(
129         static_image_mode=False,
130         max_num_hands=1,
131         model_complexity=1,
132         min_detection_confidence=0.6,
133         min_tracking_confidence=0.6
134     ) as hands:
135
136         while True:
137             # If STOPPED: do not open camera
138             if not running:
139                 latest_label = "STOPPED"
140                 frame = blank_frame(text="STOPPED - Click Start")
141                 ok, buf = cv2.imencode(".jpg", frame)
142                 if ok:
143                     yield (b"--frame\r\n"
144                           + b"Content-Type: image/jpeg\r\n\r\n" + buf.tobytes() + b"\r\n")
145                 time.sleep(0.08)
146                 continue
147

```

```

gesture_recognition.py .\  gesture_recognition.py C:\...Downloads X  index.php
C: > Users > Khae > Downloads > gesture_recognition.py > generate_frames
125 def generate_frames():
147
148     # If RUNNING: open camera
149     with lock:
150         ensure_camera_open()
151
152     # If camera failed to open (permissions / busy / backend)
153     if cap is None:
154         latest_label = "CAMERA PERMISSION / BUSY"
155         frame = blank_frame(text="CAMERA PERMISSION / BUSY")
156         ok2, buf2 = cv2.imencode(".jpg", frame)
157         if ok2:
158             yield (b"--frame\r\n"
159                   + b"Content-Type: image/jpeg\r\n\r\n" + buf2.tobytes() + b"\r\n")
160             time.sleep(0.2)
161             continue
162
163     ok, frame = cap.read()
164
165     # If read fails
166     if not ok or frame is None:
167         latest_label = "CAMERA ERROR"
168         frame = blank_frame(text="CAMERA ERROR")
169         ok2, buf2 = cv2.imencode(".jpg", frame)
170         if ok2:
171             yield (b"--frame\r\n"
172                   + b"Content-Type: image/jpeg\r\n\r\n" + buf2.tobytes() + b"\r\n")
173             time.sleep(0.1)
174             continue
175

```



```

176     frame = cv2.flip(frame, 1)
177     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
178     res = hands.process(rgb)
179
180     label = "NO HAND"
181     if res.multi_hand_landmarks:
182         hand_landmarks = res.multi_hand_landmarks[0]
183         label = recognize_gesture(hand_landmarks)
184         mp_draw.draw_landmarks(frame, hand_landmarks, mp_hands.HAND_CONNECTIONS)
185
186     latest_label = label
187
188     # overlay label
189     cv2.rectangle(frame, (10, 10), (420, 70), (0, 0, 0), -1)
190     cv2.putText(frame, f"Gesture: {latest_label}", (20, 55),
191                 cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255, 255, 255), 2)
192
193     ok3, buf3 = cv2.imencode(".jpg", frame)
194     if not ok3:
195         continue
196
197     yield (b"--frame\r\n"
198           b"Content-Type: image/jpeg\r\n\r\n" + buf3.tobytes() + b"\r\n")
199
200

```

```

gesture_recognition.py \    gesture_recognition.py C:\...\Downloads \    index.php
C: > Users > Khae > Downloads > gesture_recognition.py > generate_frames

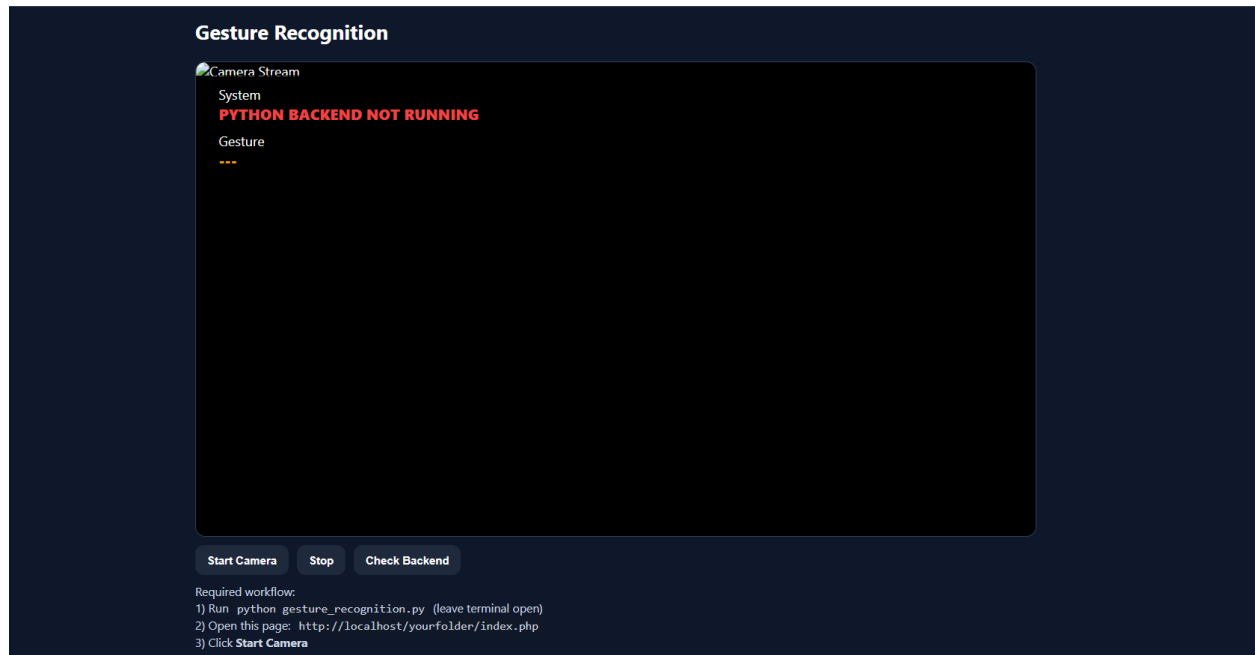
200
201 # ----- API routes -----
202 @app.get("/health")
203 def health():
204     return jsonify({"ok": True, "running": running})
205
206 @app.post("/start")
207 def start():
208     global running, latest_label
209     running = True
210     latest_label = "RUNNING"
211     return jsonify({"ok": True, "running": running})
212
213 @app.post("/stop")
214 def stop():
215     global running, latest_label
216     running = False
217     latest_label = "STOPPED"
218     with lock:
219         release_camera()
220     return jsonify({"ok": True, "running": running})
221
222 @app.get("/gesture")
223 def gesture():
224     return jsonify({"gesture": latest_label, "running": running})
225
226 @app.get("/video_feed")
227 def video_feed():
228     return Response(generate_frames(), mimetype="multipart/x-mixed-replace; boundary=frame")
229
230
231 if __name__ == "__main__":
232     # Use 127.0.0.1 for local (safe); if you access from other devices, set host="0.0.0.0"
233     app.run(host="127.0.0.1", port=5000, debug=False, threaded=True)

```

Run:

1. Terminal in that folder: `python gesture_recognition.py`
2. Browser: <http://localhost/GestureApp/index.php>

Expected Output:



Reflection Questions

1. How does gesture recognition improve or affect the way users interact with a computer compared to traditional input devices such as a mouse or keyboard?
Reflect on usability, convenience, and accessibility.
2. What user interface elements in the web application (such as system status, gesture display, or buttons) helped you understand what the system was doing?
Explain how feedback and visibility of system status influenced your experience.
3. If you were to redesign this application for real users, what improvements would you suggest to make the interaction more intuitive, efficient, or user-friendly?
Consider layout, feedback messages, controls, or accessibility.

Submission Requirements

Submit **one (1) PDF file** containing the following:

- Complete **source code** of your project.
- **Screenshot(s) of your actual output** (your own work only ---submissions identical to classmates will not be accepted).
- Include the **Google Drive (GDrive) link** of your complete project folder with accessible permissions.

Note:

Once you have completed your submission and verified that all files are included, **inform or call your professor for checking and validation.**